

From Logistic Regression to Neural Networks

Nirjhor Datta

BRAC University

From Linear Regression to Classification

Linear Regression:

$$\hat{y} = w^T x + b$$

From Linear Regression to Classification

Linear Regression:

$$\hat{y} = w^T x + b$$

Problem:

- Output is unbounded
- Not suitable for probabilities

From Linear Regression to Classification

Linear Regression:

$$\hat{y} = w^T x + b$$

Problem:

- Output is unbounded
- Not suitable for probabilities

Goal:

Predict probability: $P(y = 1|x)$

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- Output between (0,1)
- Smooth and differentiable

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- Output between (0,1)
- Smooth and differentiable

Model:

$$P(y = 1|x) = \sigma(w^T x + b)$$

$$P(y = 1|x) = \sigma(z)$$

$$P(y = 1|x) = \sigma(z)$$

$$P(y = 0|x) = 1 - \sigma(z)$$

$$P(y = 1|x) = \sigma(z)$$

$$P(y = 0|x) = 1 - \sigma(z)$$

Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Cross-Entropy Loss

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Why not MSE?

- MSE leads to slow learning
- Cross-entropy aligns with probability theory

Cross-Entropy Loss

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Why not MSE?

- MSE leads to slow learning
- Cross-entropy aligns with probability theory

Insight:

Maximizing likelihood = minimizing cross-entropy

Gradient Descent for Logistic Regression

Prediction:

$$\hat{y} = \sigma(w^T x + b)$$

Gradient Descent for Logistic Regression

Prediction:

$$\hat{y} = \sigma(w^T x + b)$$

Gradient:

$$\frac{\partial \mathcal{L}}{\partial w} = (\hat{y} - y)x$$

Gradient Descent for Logistic Regression

Prediction:

$$\hat{y} = \sigma(w^T x + b)$$

Gradient:

$$\frac{\partial \mathcal{L}}{\partial w} = (\hat{y} - y)x$$

$$\frac{\partial \mathcal{L}}{\partial b} = (\hat{y} - y)$$

Gradient Descent for Logistic Regression

Prediction:

$$\hat{y} = \sigma(w^T x + b)$$

Gradient:

$$\frac{\partial \mathcal{L}}{\partial w} = (\hat{y} - y)x$$

$$\frac{\partial \mathcal{L}}{\partial b} = (\hat{y} - y)$$

Update:

$$w = w - \eta(\hat{y} - y)x$$

Key Insight

Logistic regression = single neuron

Key Insight

Logistic regression = single neuron

$$\hat{y} = \sigma(w^T x + b)$$

Logistic regression = single neuron

$$\hat{y} = \sigma(w^T x + b)$$

Idea:

- Linear transformation + non-linearity

What if we stack neurons?

What if we stack neurons?

$$a^{(1)} = \sigma(W^{(1)}x + b^{(1)})$$

$$a^{(2)} = \sigma(W^{(2)}a^{(1)} + b^{(2)})$$

What if we stack neurons?

$$a^{(1)} = \sigma(W^{(1)}x + b^{(1)})$$

$$a^{(2)} = \sigma(W^{(2)}a^{(1)} + b^{(2)})$$

Now we can model complex functions

Artificial Neural Network (ANN)

Structure:

- Input layer
- Hidden layers
- Output layer

Artificial Neural Network (ANN)

Structure:

- Input layer
- Hidden layers
- Output layer

Each layer:

$$a = \sigma(Wx + b)$$

Activation Functions

- ReLU: $\max(0, x)$
- Sigmoid
- Tanh

Activation Functions

- ReLU: $\max(0, x)$
- Sigmoid
- Tanh

Why needed?

- Introduce non-linearity

Forward Pass

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \sigma(z^{(l)})$$

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \sigma(z^{(l)})$$

Final output:

$$\hat{y} = a^{(L)}$$

Loss Function (Neural Network)

$$\mathcal{L} = - \sum y \log(\hat{y})$$

Loss Function (Neural Network)

$$\mathcal{L} = - \sum y \log(\hat{y})$$

Goal:

$$\min \mathcal{L}$$

Backpropagation (Intuition)

Apply chain rule layer by layer

Backpropagation (Intuition)

Apply chain rule layer by layer

Key idea:

- Error flows backward
- Each layer updates its weights

Backpropagation (Intuition)

Apply chain rule layer by layer

Key idea:

- Error flows backward
- Each layer updates its weights

Form:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W}$$

Neural networks = stacked logistic regressions

Neural networks = stacked logistic regressions

- Linear regression $\rightarrow$ simple patterns
- Logistic regression $\rightarrow$ classification
- Neural networks $\rightarrow$ complex functions

Takeaway

- Logistic regression outputs probabilities
- Neural networks generalize this idea
- Learning = minimizing loss via gradient descent